AI for .NET Developers

This course is designed for .NET developers who want to build real-world AI-powered applications using **C#, ASP.NET Core Web API, Microsoft.Extensions.AI, OpenAI, Azure OpenAI, Microsoft Foundry, Semantic Kernel, Microsoft Agent Framework, MCP, Vector Databases, RAG, ML.NET, Foundry Local, SQL Server, Docker, Azure Container Registry (ACR), Azure Deployment, Azure DevOps, Azure Pipelines, AI Evaluation, Monitoring, and CI/CD**.

The course starts with AI fundamentals and gradually moves toward **production-ready AI application development, covering** security, scalability, structured outputs, tool calling, Retrieval-Augmented Generation, AI agents, evaluation, observability, cost optimization, deployment, and industry-level projects.

Contact Us

Telegram Group for more information: https://telegram.me/dotnettutorials
Mobile / WhatsApp Number: +91 70218 01173
WhatsApp Group: https://chat.whatsapp.com/LKjCuD8dFSK9MrvEWhKali
YouTube Channel: Click here to join

Target Audience

- ASP.NET Core Developers
- C# Developers
- Backend Developers
- Full Stack .NET Developers
- Web API Developers
- Software Architects
- Team Leads
- Developers who want to build AI-powered business applications

Prerequisites

- Basic C# programming knowledge
- ASP.NET Core Web API fundamentals
- REST and HTTP fundamentals
- Dependency Injection basics
- Basic asynchronous programming knowledge in C#
- Basic SQL Server knowledge
- Basic cloud knowledge is helpful but not mandatory
- No prior AI or Machine Learning experience is required

Course Details:

Date: September 5, 2026, Every Saturday and Sunday
Time: 7:00 AM – 10:00 AM, IST
Duration: 4 Months
Course Details Syllabus: https://dotnettutorials.net/lesson/ai-for-net-developers-online-training/
Registration Link: https://forms.gle/L6CiFfd8mNoELEwB6

Module 1: AI Foundations and the .NET AI Ecosystem

Build a strong conceptual foundation in Artificial Intelligence and the modern .NET AI ecosystem. By the end of this module, participants will understand how AI, Machine Learning, Generative AI, Large Language Models, model selection, secure provider configuration, prompt engineering, and context engineering fit together before they begin building production AI applications.

Chapter 1.1: AI, Machine Learning, Deep Learning, and Generative AI

This chapter establishes the foundation of the course. You will learn how Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI differ; how deterministic software differs from probabilistic AI; the major learning paradigms and ML problem types; what foundation models, LLMs, and SLMs are; and when AI is or is not an appropriate solution.

Highlighted Topics

- Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI
- Traditional deterministic software vs probabilistic AI software
- Predictive AI vs Generative AI
- Supervised, unsupervised, and reinforcement learning overview
- Classification, regression, clustering, and anomaly detection concepts
- Neural networks at a developer-friendly level
- Foundation models, LLMs, and SLMs
- Common enterprise AI patterns: generation, extraction, classification, summarization, search, RAG, tools, and agents
- When AI should and should not be used

Chapter 1.2: Large Language Models, Transformers, Tokens, Context, and Hallucinations

This chapter explains what happens inside modern language-model systems at a developer-friendly level. You will learn the role of transformers and attention, how models are trained and used for inference, how tokens and context windows affect requests, why model responses are probabilistic, why hallucinations occur, and how grounding and constraints improve reliability.

Highlighted Topics

- Large Language Models and Natural Language Processing fundamentals
- Transformer architecture at a high level
- Attention and self-attention concepts
- Pre-training, instruction tuning, alignment, and inference
- Tokens, tokenization, input tokens, output tokens, and reasoning tokens
- Context windows, context limits, and token budgets
- Parametric knowledge vs external knowledge
- Why LLM responses are probabilistic
- Why hallucinations occur
- Grounding, constraints, and response reliability

Chapter 1.3: AI Models, Inference Controls, and Model Selection

This chapter helps you choose models intentionally rather than selecting them only by name or popularity. You will compare text, reasoning, embedding, vision, audio, multimodal, cloud, local, small, and large models; understand important inference controls and performance trade-offs; and decide when prompting, RAG, fine-tuning, or traditional Machine Learning is the right approach.

Highlighted Topics

- Text-generation, reasoning, embedding, vision, audio, and multimodal models
- Small vs large models and general-purpose vs specialized models
- Cloud-hosted vs local and on-device models
- OpenAI, Azure OpenAI, Microsoft Foundry, and local model families
- Context-window limits, output-token limits, and model capabilities
- Temperature, top-p, response length, and other inference controls
- Latency, throughput, time-to-first-token, rate limits, quality, cost, and privacy trade-offs
- Model snapshots and versioning
- Fallback models and capability-based routing
- Prompting vs RAG vs fine-tuning vs traditional Machine Learning

Chapter 1.4: AI Development Environment and Secure Provider Setup

This chapter prepares the complete development environment for the rest of the course. You will configure .NET 10, Visual Studio or VS Code, OpenAI, Azure OpenAI, and Microsoft Foundry; learn secure secret and identity options including Microsoft Entra ID, DefaultAzureCredential, Managed Identity, Azure RBAC, and Key Vault; use Dependency Injection, the Options pattern, and HttpClientFactory; and understand the major Microsoft AI libraries used throughout the program.

Highlighted Topics

- .NET 10 SDK, Visual Studio, and Visual Studio Code
- Console, ASP.NET Core Web API, and Worker Service project types
- OpenAI and Azure OpenAI account/configuration basics
- Microsoft Foundry projects, endpoints, and deployments
- OpenAI .NET SDK, Azure SDKs, and AI-related NuGet packages
- Microsoft Entra ID authentication and secretless-access concepts
- Azure RBAC, least-privilege role assignments, and access-control fundamentals
- Dependency Injection, Options pattern, and HttpClientFactory
- appsettings.json, environment variables, Secret Manager, and Azure Key Vault
- DefaultAzureCredential and Managed Identity basics
- Timeouts, cancellation, retries, and package-version compatibility
- Microsoft.Extensions.AI, Microsoft.Extensions.DataIngestion, Microsoft.Extensions.VectorData, Microsoft.Extensions.AI.Evaluation, Microsoft Agent Framework, MCP C# SDK, .NET Aspire, and ML.NET ecosystem overview

Chapter 1.5: Prompt Engineering and Context Engineering

This chapter teaches how to communicate with AI systems clearly and reliably. You will learn zero-shot, one-shot, and few-shot prompting, system and user instructions, reusable templates, context ordering and token budgeting, prompt chaining, reasoning-aware design, prompt-injection awareness, and the practices required to version, test, review, and improve prompts over time.

Highlighted Topics

- Prompt engineering vs context engineering
- System/application instructions vs user input
- Zero-shot, one-shot, and few-shot prompting
- Role, task, constraints, examples, output format, and acceptance criteria
- Prompt templates, variables, and reusable prompt libraries
- Context ordering, relevance, truncation, and token budgeting
- Prompt chaining and task decomposition
- Reasoning-aware prompt design without depending on hidden reasoning
- Prompt injection-aware prompt construction
- Prompt versioning, testing, regression, review, rollback, and anti-patterns

Module 2: Core Generative AI Development with .NET

Develop the core skills required to integrate Generative AI into .NET and ASP.NET Core applications. This module focuses on provider SDKs and abstractions, Azure identity and secure access, reliable structured outputs, conversation state, streaming, tool calling, and production-oriented API architecture so that participants can move beyond simple prompt-and-response demos.

Chapter 2.1: OpenAI .NET SDK and Responses API

This chapter introduces direct OpenAI integration from .NET using the official SDK and the Responses API. You will learn the request/response lifecycle, SDK versus REST integration, multimodal inputs, multi-turn state, streaming and background responses, and the production concerns around cancellation, timeouts, retries, identifiers, usage metadata, and errors.

Highlighted Topics

- OpenAI API concepts and authentication
- Official OpenAI .NET SDK
- Responses API request/response lifecycle
- Responses API vs older chat-completion patterns
- SDK-based integration vs raw REST using HttpClient
- Text, image, and file inputs
- Multi-turn conversation patterns and provider-side state
- Streaming responses and client disconnect handling
- Background responses and long-running operations
- CancellationToken, timeouts, retries, backoff, request identifiers, token usage, and error handling

Chapter 2.2: Azure OpenAI with ASP.NET Core

This chapter moves the same core AI development concepts into the Azure ecosystem. You will learn Azure OpenAI architecture, model deployments and endpoints, API key versus Microsoft Entra authentication, Managed Identity, the Azure OpenAI Responses API, streaming, structured outputs, tools, content safety, networking, and enterprise configuration considerations.

Highlighted Topics

- Azure OpenAI architecture and service model
- Azure OpenAI vs OpenAI
- Model deployments and endpoints
- API key and Microsoft Entra authentication
- Managed Identity fundamentals
- Azure OpenAI Responses API
- Streaming and multi-turn conversations
- Structured outputs and tool support
- Enterprise security, networking, and data considerations
- Content safety integration and deployment configuration

Chapter 2.3: Azure Identity, Access Control, and Secretless Security for AI Applications

This chapter establishes the identity and access-control model required for enterprise Azure AI applications. You will learn how Microsoft Entra ID, DefaultAzureCredential, system-assigned and user-assigned Managed Identities, Azure RBAC, least-privilege permissions, and Azure Key Vault work together so applications can access Azure OpenAI, Microsoft Foundry, Azure AI Search, storage, and other Azure resources without embedding long-lived credentials in code or configuration.

Highlighted Topics

- Microsoft Entra ID authentication for Azure AI applications
- DefaultAzureCredential for local development and Azure authentication flows
- Managed Identity for Azure-hosted .NET applications
- System-assigned vs user-assigned Managed Identities
- Azure RBAC and least-privilege role assignments
- Role-based access to Azure OpenAI, Microsoft Foundry, Azure AI Search, storage, and Key Vault
- Azure Key Vault for secrets, keys, and secure configuration
- Secretless authentication and avoiding credentials in source code and appsettings files
- Development identity vs production identity
- Authentication failures, authorization failures, and secure troubleshooting principles

Chapter 2.4: Microsoft.Extensions.AI and Provider-Independent Architecture

This chapter teaches provider-independent AI architecture in .NET. You will work with Microsoft.Extensions.AI abstractions such as IChatClient and IEmbeddingGenerator, understand client pipelines and middleware, switch between cloud and local providers,

design testable application services, and learn where direct provider SDK usage still makes sense.

Highlighted Topics

- Microsoft.Extensions.AI abstractions for .NET applications
- IChatClient and IEmbeddingGenerator
- Chat messages, chat options, and streaming abstractions
- Dependency Injection registration patterns
- Decorators, pipelines, middleware, caching, and telemetry concepts
- Automatic function invocation concepts
- Switching between OpenAI, Azure OpenAI, Ollama, and compatible providers
- Testing application services behind AI abstractions
- When direct provider SDK usage is appropriate
- Architecture boundary: Controller -> Application Service -> AI Client

Chapter 2.5: Structured Outputs and Strongly Typed AI Contracts

This chapter focuses on turning probabilistic model responses into dependable application contracts. You will learn JSON Schema, strongly typed DTOs, schema-constrained outputs, deserialization, semantic and business validation, malformed or blocked-output handling, response-contract versioning, and why valid JSON must still be treated as untrusted input.

Highlighted Topics

- Risks of free-form text parsing in production applications
- JSON output vs schema-constrained structured output
- Designing DTOs for AI-generated data
- JSON Schema fundamentals
- Required fields, enums, nested objects, arrays, and nullable values
- Deserialization and schema validation
- Semantic and business-rule validation after schema validation
- Handling partial, malformed, or policy-blocked outputs
- Versioning AI response contracts
- Never blindly trusting valid JSON

Chapter 2.6: Conversation State, Memory, Context Management, and Streaming

This chapter explains how conversational applications maintain useful state without allowing context to grow without control. You will learn stateless and stateful designs, short-term context and durable memory, SQL Server persistence, summarization and sliding-window strategies, memory versus RAG retrieval, SSE and SignalR streaming, privacy controls, and multi-tenant isolation.

Highlighted Topics

- Stateless vs stateful conversational applications
- Conversation history, sessions, and multi-turn interactions
- Short-term context vs durable memory
- User-profile memory vs conversation memory
- SQL Server-based conversation persistence

- Summarization, compaction, sliding-window, and token-budget strategies
- Memory retrieval vs RAG retrieval
- Streaming with Server-Sent Events and SignalR
- Consent, deletion, retention, and privacy controls
- Multi-tenant isolation, concurrency, and deciding what not to remember

Chapter 2.7: Function Calling, Tool Calling, and Safe Action Execution

This chapter shows how AI moves from generating text to safely invoking real application capabilities. You will learn function and tool schemas, C# and service integration, database and external API tools, multi-tool and parallel calls, validation and authorization, human approval, idempotency, least privilege, retries, timeouts, call limits, and compensation strategies.

Highlighted Topics

- Function calling vs tool calling
- AI response generation vs AI-triggered actions
- Defining tool schemas and arguments in .NET
- Calling C# methods, business services, databases, and external APIs
- Tool-selection loops and multi-tool conversations
- Parallel tool calls
- Validating and authorizing tool inputs and results
- Human approval for sensitive actions
- Idempotency, duplicate-action prevention, and least-privilege tool design
- Failure handling, timeouts, retries, maximum tool-call limits, and compensation strategies

Chapter 2.8: Production Architecture for AI-Powered ASP.NET Core APIs

This chapter brings the preceding concepts together into a production-ready ASP.NET Core architecture. You will learn layered design, synchronous and streaming endpoints, Controllers and Minimal APIs, DTOs and services, authentication, rate limiting, correlation and auditing, background queues, long-running jobs, health checks, dead-letter handling, and safe transaction boundaries.

Highlighted Topics

- Layered and Clean Architecture considerations for AI applications
- Synchronous, streaming, and asynchronous AI endpoint design
- Controllers, Minimal APIs, DTOs, service abstraction, and dependency inversion
- Global exception handling, Problem Details, and safe client errors
- Authentication, authorization, rate limiting, and per-user quotas
- Request correlation, usage auditing, cancellation, and health checks
- BackgroundService and queue-based AI processing
- Long-running jobs, status endpoints, cancellation, idempotency, and dead-letter handling
- SSE vs SignalR vs standard HTTP streaming
- Avoiding model calls inside database transactions

Module 3: Microsoft Foundry, Applied AI Services, and Multimodal AI

Understand Microsoft enterprise AI services and multimodal application capabilities. Participants will learn how Microsoft Foundry manages models and enterprise AI resources, and how to incorporate vision, OCR, Document Intelligence, speech, language, content safety, multimodal inputs, and real-time voice experiences into .NET solutions.

Chapter 3.1: Microsoft Foundry and Enterprise Model Management

This chapter introduces Microsoft Foundry as the enterprise platform for organizing AI projects, models, deployments, credentials, environments, and governance. You will learn how model catalogs and lifecycle management work, how Microsoft identity integrates with Foundry resources, and how observability, evaluation, and Responsible AI fit into enterprise model operations.

Highlighted Topics

- Microsoft Foundry architecture
- Foundry projects, resources, endpoints, and credentials
- Model catalog and model deployments
- OpenAI and non-OpenAI model choices
- Azure authentication and Managed Identity
- Environment and project organization
- Model configuration, lifecycle, and version management
- Foundry observability and evaluation overview
- Enterprise governance and responsible AI considerations

Chapter 3.2: Vision, OCR, and Document Intelligence

This chapter teaches how .NET applications can understand images and business documents. You will learn computer-vision fundamentals, OCR, Document Intelligence, layout and structured extraction, invoice and form processing, confidence-based validation, human review, and how to combine extracted document data with Generative AI workflows.

Highlighted Topics

- Computer vision and image-understanding fundamentals
- OCR workflows
- Document Intelligence
- Layout and structured document extraction
- Invoice, form, and document-image processing
- Confidence scores and validation
- Human review for uncertain extraction
- Combining extracted document data with Generative AI

Chapter 3.3: Speech, Audio, Translator, and Language Services

This chapter expands AI applications beyond text-only interactions. You will learn speech-to-text, text-to-speech, audio concepts, translation, sentiment analysis, language detection, key phrases, entity extraction, and PII detection, together with

patterns for combining these applied language and speech services with Generative AI.

Highlighted Topics

- Speech-to-text
- Text-to-speech
- Audio processing concepts
- Language translation
- Sentiment analysis
- Language detection
- Key phrase and entity extraction
- PII detection concepts
- Combining language services with Generative AI

Chapter 3.4: Content Safety, Moderation, and PII Protection

This chapter focuses on protecting users and enterprise data during AI interactions. You will learn how to moderate inputs and outputs, detect harmful or sensitive content, identify and mask PII, block unsafe responses, configure safety thresholds, log moderation decisions, and incorporate human review and escalation into higher-risk workflows.

Highlighted Topics

- Input moderation and output moderation
- Toxicity and harmful-content detection
- Sensitive-content handling
- PII detection and masking
- Unsafe response blocking
- Safety policies and thresholds
- Moderation logging and audit requirements
- Human review and escalation

Chapter 3.5: Multimodal AI Applications

This chapter teaches how to design applications that combine multiple input and output modalities. You will work with text, images, audio, documents, and files; understand image question answering and document-image processing; generate structured multimodal results; and apply multimodal prompting, privacy, and safety practices.

Highlighted Topics

- Multimodal AI concepts
- Text, image, audio, document, and file inputs
- Image question answering
- Document image processing
- Multimodal structured extraction
- Combining text and vision
- Multimodal prompt design
- Safety and privacy considerations for multimodal data

Chapter 3.6: Realtime and Voice AI

This chapter introduces low-latency conversational and voice experiences. You will learn realtime AI architecture, streaming audio, speech pipelines, WebSocket-based interaction, voice-agent design, turn detection, interruptions, latency optimization, realtime tool calling, and privacy and safety considerations for voice applications.

Highlighted Topics

- Realtime AI architecture
- Streaming audio
- Speech-to-text and text-to-speech pipelines
- WebSocket-based AI interactions
- Voice agent architecture
- Turn detection concepts
- Interruptions and conversational control
- Latency optimization
- Realtime tool calling concepts
- Voice safety and privacy

Module 4: Embeddings, Vector Search, and Retrieval-Augmented Generation

Master the complete knowledge-grounding pipeline behind semantic search and Retrieval-Augmented Generation. Participants will learn embeddings, ingestion, chunking, vector stores, advanced Azure AI Search, search strategies, foundational and advanced RAG, end-to-end Azure AI Search RAG workflows, enterprise RAG security and freshness, and systematic RAG evaluation for reliable knowledge-aware applications.

Chapter 4.1: Embeddings and Semantic Search

This chapter introduces embeddings as the foundation of semantic retrieval. You will learn how text becomes vectors, how document and query embeddings differ, how similarity is calculated, how semantic search differs from keyword search, and how metadata, ranking, model selection, batching, reuse, versioning, and cost affect an embedding solution.

Highlighted Topics

- What embeddings are and why vector representations matter
- Text-to-vector conversion
- Embedding dimensions and embedding model selection
- Document and query embeddings
- Cosine similarity, dot-product similarity, and distance concepts
- Semantic search vs keyword search
- Metadata and filters
- Search ranking and relevance
- Embedding versioning, reuse, batching, and cost optimization
- IEmbeddingGenerator with Microsoft.Extensions.AI

Chapter 4.2: AI Data Ingestion, Chunking, and Knowledge Indexing

This chapter explains how raw enterprise knowledge becomes searchable AI-ready content. You will learn ingestion architecture, document readers, normalization, multiple chunking strategies, overlap and metadata design, embedding generation, batch indexing, duplicate detection, incremental updates, deletion propagation, index versioning, and re-indexing.

Highlighted Topics

- AI data-ingestion architecture
- Microsoft.Extensions.DataIngestion concepts
- Document readers and normalized document models
- PDF, Word, Markdown, text, HTML, and web/API content
- Text cleaning and normalization
- Fixed-size, token-aware, sentence, header-aware, semantic, and hierarchical chunking concepts
- Chunk overlap, boundaries, metadata enrichment, and document identifiers
- Embedding generation and batch indexing
- Duplicate detection and content hashing
- Incremental indexing, updates, deletion propagation, index versioning, and re-indexing

Chapter 4.3: Vector Stores and Microsoft.Extensions.VectorData

This chapter teaches the role of vector stores in semantic retrieval applications. You will learn vector data models, collections, CRUD operations, vector and text search, hybrid search, metadata filters, provider-independent architecture with Microsoft.Extensions.VectorData, and the practical roles of Azure AI Search, SQL Server vector search, Qdrant, and other providers.

Highlighted Topics

- Vector stores and vector databases
- Microsoft.Extensions.VectorData abstraction
- Collections, records, keys, data properties, and vector properties
- CRUD operations for vector records
- Vector search, text search, and hybrid search
- Metadata filtering
- Provider-independent vector-store architecture
- Azure AI Search
- SQL Server vector search
- Qdrant and other vector-database options

Chapter 4.4: Advanced Azure AI Search for Enterprise Retrieval

This chapter provides a focused Azure AI Search deep dive for production retrieval systems. You will learn how indexes, schemas, data sources, indexers, skillsets, integrated chunking and vectorization, vector fields and vectorizers, HNSW-based vector search, semantic configuration, hybrid search, semantic ranking, filters, and index refresh strategies work together to create secure and maintainable enterprise search foundations for RAG.

Highlighted Topics

- Azure AI Search service architecture and search-index design

- Search indexes, fields, schemas, searchable/filterable/retrievable properties
- Data sources and indexers
- Skillsets and AI enrichment pipelines
- Text Split skill and document chunking during indexing
- Azure OpenAI Embedding skill and embedding generation
- Integrated vectorization for indexing and query-time vectorization
- Vector fields, vector profiles, and vectorizers
- HNSW and vector-search configuration concepts
- Keyword/BM25, vector, semantic, and hybrid search in Azure AI Search
- Semantic configuration, semantic ranker, and reranking
- Reciprocal Rank Fusion in hybrid retrieval
- Metadata filters and retrieval constraints
- Indexer scheduling, incremental updates, freshness, and re-indexing
- Keyless authentication, Azure RBAC, and security-aware search access

Chapter 4.5: Search Strategies for AI Applications

This chapter compares the major retrieval strategies used by modern AI applications. You will learn keyword and BM25 search, semantic and vector search, hybrid search, metadata filtering, semantic ranking and reranking, Reciprocal Rank Fusion, Maximal Marginal Relevance, Top-K and threshold tuning, diversity, relevance, and retrieval failure analysis.

Highlighted Topics

- Keyword search and BM25 concepts
- Semantic search
- Vector search
- Hybrid search
- Metadata filtering
- Semantic ranking and reranking
- Reciprocal Rank Fusion concepts
- Maximal Marginal Relevance concepts
- Top-K, dynamic Top-K, and threshold tuning
- Search-result diversity, relevance, and failure analysis

Chapter 4.6: Retrieval-Augmented Generation Fundamentals

This chapter builds the conceptual and architectural foundation for Retrieval-Augmented Generation. You will learn the complete RAG lifecycle from ingestion and embeddings to retrieval, context construction, grounding, answer generation, citations, no-answer handling, and hallucination reduction, and you will compare RAG with long-context prompting and fine-tuning.

Highlighted Topics

- What RAG is and why it is needed
- RAG architecture and request lifecycle
- Ingestion, retrieval, and generation pipelines
- Document chunking, embedding generation, and vector storage
- Query embedding, Top-K retrieval, and metadata filters
- Context construction and prompt grounding

- Grounded responses and source citations
- No-answer and insufficient-context handling
- Reducing hallucinations with evidence
- RAG vs long context vs fine-tuning

Chapter 4.7: Enterprise RAG Workflows with Azure AI Search

This chapter connects the individual RAG concepts into a complete Azure enterprise workflow. You will learn how source documents flow through Azure AI Search data sources, indexers, skillsets, chunking, embeddings, and vector indexes; how user questions are executed with vector, hybrid, and semantic retrieval; and how retrieved evidence is passed to an LLM to generate grounded answers with citations, authorization filters, freshness controls, and measurable quality.

Highlighted Topics

- End-to-end Azure RAG architecture and request lifecycle
- Enterprise content source to Azure AI Search ingestion pipeline
- Indexer + skillset + chunking + embedding + vector-index workflow
- Azure OpenAI or Microsoft Foundry embedding integration
- Query-time vectorization and query embedding
- Vector search, hybrid search, and semantic ranking workflow
- Metadata filters and security trimming before grounding
- Top-K retrieval, relevance thresholds, and context construction
- Passing retrieved evidence to the Generative AI model
- Grounded answer generation with source citations
- No-answer handling when reliable evidence is unavailable
- Document freshness, index refresh, and update propagation
- RAG telemetry, evaluation, and retrieval-quality validation

Chapter 4.8: Advanced RAG and Retrieval Engineering

This chapter moves beyond basic RAG into retrieval engineering. You will learn query rewriting, expansion, decomposition, multi-query retrieval, hybrid retrieval and reranking, parent-child retrieval, contextual chunking, context compression and packing, lost-in-the-middle mitigation, source routing, citation verification, caching, and agentic retrieval concepts.

Highlighted Topics

- Query rewriting and query expansion
- Multi-query retrieval
- Query decomposition
- Hybrid retrieval and semantic reranking
- Parent-child and hierarchical retrieval
- Contextual and semantic chunking concepts
- Context compression, relevance filtering, and context packing
- Lost-in-the-middle mitigation
- Multi-source retrieval and knowledge-source routing
- Citation verification, retrieval caching, and agentic retrieval concepts

Chapter 4.9: Enterprise RAG Security, Freshness, and Knowledge Governance

This chapter focuses on the requirements that make RAG suitable for enterprise and multi-tenant use. You will learn authorization-aware retrieval, ACLs and security trimming, tenant isolation, metadata security, PII controls, freshness and versioning, deletion propagation, indirect prompt-injection risks, source trust, audit trails, citation traceability, and cross-tenant leakage prevention.

Highlighted Topics

- Document-level and chunk-level authorization
- Authorization-aware retrieval and security trimming
- ACL-aware retrieval, tenant isolation, and user-specific knowledge access
- Metadata security filters
- PII and confidential-data controls
- Freshness, document versioning, re-indexing, and stale-content management
- Source deletion and embedding deletion propagation
- Indirect prompt injection through retrieved documents
- Trusted vs untrusted source handling
- Retrieval audit trails, citation traceability, and preventing cross-tenant leakage

Chapter 4.10: RAG Evaluation and Quality Improvement

This chapter teaches how to measure and systematically improve a RAG system rather than relying only on manual impressions. You will learn retrieval versus generation quality, groundedness, relevance, completeness, citation correctness, precision and recall concepts, golden datasets, controlled experiments, diagnostics, failure analysis, and RAG regression testing.

Highlighted Topics

- Retrieval quality vs generation quality
- Groundedness, relevance, completeness, and citation correctness
- Context precision and context recall concepts
- Retrieval precision, recall, hit rate, and ranking quality
- Golden evaluation datasets
- Chunk-size, Top-K, threshold, prompt, and model experiments
- Retrieval diagnostics and failure analysis
- RAG regression testing

Module 5: AI Orchestration, Agents, MCP, and Interoperability

Learn how to design and build agentic AI systems responsibly using the .NET ecosystem. This module progresses from AI agent fundamentals and agent creation with Microsoft Agent Framework to tools, memory, production agent architecture, guardrails, durable workflows, MCP integration, multi-agent coordination, human oversight, interoperability, hosting, and Semantic Kernel concepts for existing applications.

Chapter 5.1: AI Agent Fundamentals

This chapter establishes the core mental model for agentic AI. You will learn the difference between assistants, tools, workflows, and agents; understand agent loops, goals, state, context, memory, planning, and autonomy; compare deterministic workflows with autonomous behavior; and recognize the reliability, cost, safety, and use-case limits of agents.

Highlighted Topics

- AI assistant vs AI agent
- Workflow vs agent
- Tool calling vs agentic behavior
- Goal-directed execution
- Agent loop, state, context, memory, planning, and tools
- Autonomous vs controlled agents
- Deterministic workflows vs autonomous decisions
- Agent limitations, cost, reliability, and safety
- When agents should not be used

Chapter 5.2: Microsoft Agent Framework and Agent Creation

This chapter introduces Microsoft Agent Framework for creating modern agent applications in .NET. You will learn how to create and configure agents with models, instructions, tools, sessions, context, state, persistence, providers, and middleware; connect agents to application services and knowledge; expose agents as tools; and use lifecycle hooks, observability, human approval, and authorization boundaries to support production use.

Highlighted Topics

- Microsoft Agent Framework architecture and concepts
- Creating and configuring agents
- Agent instructions, models, tools, and sessions
- Agent context, state, and persistence concepts
- Context providers and middleware concepts
- Agents as tools
- Lifecycle hooks and application integration
- Error handling and observability
- Human approval integration
- Agent safety and authorization boundaries

Chapter 5.3: Agent Tools, Memory, Context, and Knowledge

This chapter explores the internal capabilities that make an agent useful over longer tasks. You will learn function- and tool-connected agents, short-term and persistent memory, context providers, knowledge retrieval, state management, context-window optimization, tool permission boundaries, safe execution, and human-in-the-loop patterns.

Highlighted Topics

- Function-connected and tool-connected agents
- Short-term vs persistent agent memory

- Context providers
- Knowledge retrieval for agents
- Tool permission boundaries
- State management
- Context-window optimization
- Safe tool execution
- Human-in-the-loop agent interactions

Chapter 5.4: Production AI Agent Architecture, Guardrails, Reliability, and Cost Control

This chapter focuses on the architecture required to operate AI agents safely and predictably in production. You will learn how to persist state, control autonomy, apply input/output/tool guardrails, enforce authorization and tool allow-lists, define termination conditions and call limits, handle failures and retries, provide graceful fallback behavior, control token and cost budgets, require human approval for high-impact actions, and trace agent decisions and tool execution for operational accountability.

Highlighted Topics

- Production AI agent architecture and service boundaries
- Agent sessions, state persistence, and recovery
- Deterministic controls around autonomous agent behavior
- Input, output, and tool-execution guardrails
- Authorization guardrails, tool allow-lists, and least privilege
- Maximum iterations, maximum tool calls, and termination conditions
- Timeouts, retries, fallback behavior, and graceful degradation
- Failure handling, compensation, and recovery strategies
- Idempotency and safe handling of side-effecting actions
- Human approval, escalation, and manual intervention
- Token budgets, cost limits, and per-agent usage controls
- Observability, audit trails, correlation, and agent/tool tracing

Chapter 5.5: Agent Workflows, Agent Harness, and Durable Execution

This chapter teaches how to coordinate reliable, long-running agent workflows. You will learn graph-based orchestration, sequential and parallel execution, branching and routing, event-driven flows, Agent Harness concepts, checkpoints, pause and resume, durable state, failure recovery, compensation, distributed workflows, and how to choose between a single agent, workflow, and multi-agent system.

Highlighted Topics

- Agent workflows and graph-based orchestration
- Sequential and parallel workflows
- Branching, routing, and conditional execution
- Event-driven workflows
- Agent Harness concepts
- Workflow state, checkpoints, pause/resume, and resumability
- Long-running and durable agent workflows

- Failure recovery and compensation
- Distributed workflow concepts
- Choosing a single agent vs workflow vs multi-agent system

Chapter 5.6: Model Context Protocol (MCP) for .NET Developers

This chapter introduces Model Context Protocol as a standard way for AI applications and agents to discover and use external capabilities. You will learn MCP hosts, clients, servers, tools, resources, prompts, capability discovery, the C# SDK, building MCP integrations in .NET, remote MCP, and the security, authentication, authorization, validation, and governance requirements around exposed tools.

Highlighted Topics

- What MCP is and why it matters
- MCP architecture: hosts, clients, and servers
- MCP tools, resources, and prompts
- Tool discovery and capability negotiation
- MCP C# SDK
- Building MCP clients and servers in .NET
- Exposing .NET business services through MCP
- Connecting agents and applications to MCP servers
- Remote MCP concepts
- Authentication, authorization, trust boundaries, input validation, and enterprise MCP governance

Chapter 5.7: Multi-Agent Systems, Human-in-the-Loop, Interoperability, and Hosting

This chapter extends agent design into coordinated and distributed systems. You will learn multi-agent roles, sequential and parallel collaboration, handoffs and shared state, human approval and escalation, conflict and cost controls, ASP.NET Core hosting, durable session persistence, background coordination, agent-to-agent interoperability, UI integration, streaming events, and realtime agent experiences.

Highlighted Topics

- Multi-agent architecture and specialized agent roles
- Sequential and parallel collaboration
- Agent handoffs, shared state, and coordination
- Human approval checkpoints, escalation, and manual review
- Conflict handling, cost controls, and reliability controls
- ASP.NET Core hosting for agents
- Persisting agent sessions and workflow state
- Background execution and event-driven coordination
- Agent-to-agent interoperability and A2A-style communication concepts
- Remote agent services, UI integration, streaming agent events, and realtime agent experiences

Chapter 5.8: Semantic Kernel for .NET and Existing Application Patterns

This chapter positions Semantic Kernel within the modern .NET AI ecosystem. You will learn its kernel, plugins, functions, prompt templates, memory and planner concepts, ASP.NET Core integration, and orchestration patterns, while comparing it with direct provider SDKs, Microsoft.Extensions.AI, and newer Microsoft Agent Framework approaches.

Highlighted Topics

- Semantic Kernel architecture and core concepts
- Kernel construction and dependency integration
- Plugins, native functions, and prompt functions
- Prompt templates and function calling
- Memory and planner concepts
- Workflow orchestration concepts
- Semantic Kernel with ASP.NET Core
- Semantic Kernel vs direct provider SDKs
- Semantic Kernel vs Microsoft.Extensions.AI
- Relationship with modern Agent Framework concepts and coexistence considerations

Module 6: Enterprise Data, Model Adaptation, ML.NET, and Local AI

Learn how AI can work with enterprise data and alternative model strategies. Participants will compare structured-data AI, fine-tuning, traditional Machine Learning with ML.NET, local models, and hybrid architectures so they can select the right approach for privacy, cost, reliability, business rules, and deployment requirements.

Chapter 6.1: AI with SQL Server and Structured Enterprise Data

This chapter teaches safe AI access to structured enterprise data. You will learn natural-language-to-SQL architecture, schema discovery and grounding, SQL generation and validation, read-only policies and allow-lists, limits and timeouts, role-based and row-level access, data masking, report generation, chart-ready outputs, and patterns for combining SQL with RAG and business rules.

Highlighted Topics

- AI over structured vs unstructured enterprise data
- Natural-language-to-SQL architecture and workflows
- Schema discovery, schema grounding, and controlled context
- SQL generation and query validation
- Read-only execution, SELECT-only policies, command allow-listing, row limits, and query timeouts
- Preventing destructive or harmful SQL execution
- Role-based data access and row-level-security concepts
- Data masking and sensitive-data handling
- AI reporting, analytics, result summarization, and chart-ready structured output
- Combining SQL results with RAG and business rules

Chapter 6.2: Fine-Tuning, Model Adaptation, and Distillation

This chapter explains when model adaptation is justified after prompting and RAG have reached their limits. You will learn fine-tuning use cases, dataset preparation and splits, data quality and formatting, evaluation, overfitting and drift, LoRA and parameter-efficient tuning, distillation, synthetic data, and the privacy, cost, maintenance, and governance implications of customized models.

Highlighted Topics

- When prompting and RAG are insufficient
- Fine-tuning concepts and suitable use cases
- Prompting vs RAG vs fine-tuning
- Training, validation, and holdout datasets
- Data quality, labeling, formatting, and JSONL-style datasets
- Training metrics and evaluation before and after adaptation
- Overfitting and model drift concepts
- LoRA and parameter-efficient tuning concepts
- Distillation and smaller-model strategies
- Synthetic-data generation, privacy, cost, maintenance, and governance

Chapter 6.3: Traditional Machine Learning with ML.NET and Hybrid AI

This chapter shows where traditional Machine Learning remains the better engineering choice. You will learn core ML.NET scenarios such as classification, regression, clustering, anomaly detection, recommendations, and forecasting; data preparation and evaluation; model serving from ASP.NET Core; and hybrid patterns that combine ML predictions, LLM explanations, RAG, rules, and deterministic validation.

Highlighted Topics

- Traditional Machine Learning vs Generative AI
- Classification, regression, clustering, anomaly detection, recommendation systems, and forecasting basics
- Data preparation and feature engineering fundamentals
- Model training and evaluation
- ML.NET Model Builder and CLI overview
- Serving ML.NET models from ASP.NET Core
- Combining ML.NET predictions with LLM explanations
- Rule engines plus LLM workflows
- RAG plus business rules and deterministic validation
- Choosing ML.NET, LLMs, RAG, rules, or a hybrid architecture

Chapter 6.4: Local LLMs, Foundry Local, Ollama, and On-Device AI

This chapter explores private and offline AI using local models. You will learn Foundry Local, Ollama, ONNX Runtime, model formats and quantization, hardware and CPU/GPU considerations, local embeddings and RAG, privacy-sensitive use cases, hybrid cloud/local routing, provider abstraction, and the operational limitations of running models locally.

Highlighted Topics

- Why organizations run AI models locally
- Cloud vs local vs on-device AI
- Foundry Local concepts and .NET integration
- Ollama and local model serving
- ONNX Runtime fundamentals
- Model size, formats, quantization, memory, CPU/GPU, and hardware considerations
- Offline and privacy-sensitive application scenarios
- Local embeddings and local RAG patterns
- Hybrid cloud/local architectures and provider abstraction
- Limitations and operational trade-offs of local models

Module 7: AI Security, Responsible AI, and Governance

Develop a security-first and governance-aware mindset for production AI systems. Participants will identify AI-specific threats, apply layered guardrails and Responsible AI practices, and use red teaming and security testing to reduce risks across prompts, RAG, tools, agents, sensitive data, and multi-tenant applications.

Chapter 7.1: AI Security Threats and Threat Modeling

This chapter introduces the threat landscape unique to AI applications. You will learn direct and indirect prompt injection, jailbreaks, data leakage and exfiltration, unsafe outputs and tools, excessive agency, malicious retrieved content and RAG poisoning, denial-of-service risks, threat modeling, trust boundaries, least privilege, defense in depth, and production security review practices.

Highlighted Topics

- AI-specific attack surfaces
- Direct and indirect prompt injection
- Jailbreaks and instruction-hierarchy attacks
- Sensitive-data leakage, excessive data exposure, and data exfiltration
- Unsafe output handling
- Unsafe tool execution, excessive agency, and confused-deputy risks
- Malicious documents, retrieved content, and RAG poisoning
- Model and tool denial-of-service risks
- Threat modeling for prompts, RAG, tools, agents, and data flows
- Trust boundaries, least privilege, defense in depth, and production security review

Chapter 7.2: Guardrails, Content Safety, Responsible AI, and Governance

This chapter turns AI safety and governance principles into application controls. You will learn input and output validation, content moderation, PII minimization, authorization before retrieval and actions, human approval for high-impact decisions, bias and fairness considerations, transparency, Responsible AI, auditability, privacy, compliance, governance, and safe fallback or refusal behavior.

Highlighted Topics

- Input validation and output validation
- Content moderation and policy enforcement
- PII detection, masking, and data minimization
- Authorization before retrieval and tool execution
- Human approval for high-impact actions
- Bias, fairness, transparency, and explainability concepts
- Responsible AI principles and human oversight
- Audit logging and accountability
- Privacy, compliance, retention, model/prompt governance, and AI usage policies
- Safe fallback and refusal handling

Chapter 7.3: AI Red Teaming and Security Testing

This chapter teaches how to actively test AI systems against misuse and adversarial behavior. You will design prompt-injection and jailbreak test suites, test malicious documents and RAG poisoning, simulate tool abuse and privilege escalation, check cross-tenant leakage and sensitive-data extraction, exercise agent abuse cases, and create repeatable security regression tests with documented remediation.

Highlighted Topics

- Adversarial prompt testing
- Prompt-injection test suites
- Jailbreak and policy-bypass testing
- RAG poisoning and malicious-document testing
- Tool abuse and privilege-escalation scenarios
- Cross-tenant data leakage testing
- Sensitive-data extraction attempts
- Agent workflow abuse cases
- Security regression testing
- Documenting findings and remediation

Module 8: AI Testing, Evaluation, and Quality Engineering

Learn how to test and evaluate AI systems whose outputs are probabilistic rather than fully deterministic. This module covers AI-aware testing, evaluation datasets and quality metrics, automated evaluation, regression testing, and release gates that help teams measure quality and prevent regressions across prompts, models, RAG, and agents.

Chapter 8.1: Testing AI Applications

This chapter explains how testing changes when application behavior includes non-deterministic AI outputs. You will learn to unit test deterministic application logic around AI boundaries, mock AI clients, run integration and contract tests, test structured outputs, streaming, cancellation, tool authorization, RAG pipelines, and agent workflows, and separate deterministic assertions from probabilistic quality checks.

Highlighted Topics

- Why AI testing differs from deterministic software testing

- Non-deterministic outputs
- Unit testing application logic around AI boundaries
- Mocking and faking AI clients
- Integration testing AI APIs
- Contract testing structured outputs
- Testing streaming and cancellation
- Testing tool-call validation and authorization
- Testing RAG pipelines and agent workflows
- Deterministic business-rule tests vs probabilistic AI tests

Chapter 8.2: Evaluation Datasets and AI Quality Metrics

This chapter teaches how to build reliable evaluation datasets and choose meaningful AI quality metrics. You will learn golden datasets, representative cases, expected answers and rubrics, versioning and traceability, relevance, groundedness, truthfulness, completeness, coherence, retrieval quality, citation correctness, hallucination detection, agent and tool-call metrics, and human review.

Highlighted Topics

- Golden datasets and representative test cases
- Expected-answer and rubric design
- Dataset versioning and traceability
- Relevance, groundedness, truthfulness, completeness, and coherence
- Retrieval precision, recall, ranking quality, and citation correctness
- Hallucination and unsupported-claim detection
- Task-specific evaluation metrics
- Agent evaluation and tool-call accuracy concepts
- Human evaluation and expert review
- Microsoft.Extensions.AI.Evaluation concepts

Chapter 8.3: Automated Evaluation, Regression Testing, and Release Gates

This chapter brings evaluation into the software delivery lifecycle. You will learn automated evaluation pipelines, LLM-as-judge concepts and limitations, pairwise comparisons, prompt and model change evaluation, regression baselines, safety gates, RAG answer-quality gates, cost and latency evaluation, reporting dashboards, and CI/CD release gates for AI changes.

Highlighted Topics

- Automated evaluation pipelines
- LLM-as-judge concepts and limitations
- Pairwise comparison of prompts and models
- Prompt/model change evaluation
- Prompt, model, RAG, and agent workflow regression testing
- Regression thresholds and baselines
- Safety and red-team evaluation gates
- RAG retrieval and answer-quality gates
- Cost and latency evaluation
- Evaluation reports, dashboards, and CI/CD release gates

Module 9: Observability, Performance, Resilience, and Cost Governance

Learn how to operate AI applications efficiently and reliably in production. Participants will use observability, distributed tracing, resilience patterns, caching, rate limiting, AI gateways, model routing, quotas, and cost governance to improve reliability, performance, visibility, and financial control.

Chapter 9.1: AI Observability, Logging, and Distributed Tracing

This chapter teaches how to make AI behavior visible in production. You will learn logs, metrics, traces, token and prompt telemetry, latency and dependency monitoring, correlation IDs, OpenTelemetry, Application Insights, Foundry observability, RAG source tracing, tool and agent workflow traces, user feedback, privacy-safe logging, redaction, audit retention, dashboards, and alerts.

Highlighted Topics

- AI application observability
- Logs, metrics, and traces
- Request, response, prompt, and token-usage telemetry
- Latency, error, and dependency monitoring
- Correlation IDs and distributed tracing
- OpenTelemetry concepts for AI applications
- Application Insights and Foundry observability
- RAG retrieval traces and source tracking
- Tool-call and agent-workflow traces
- User feedback, safe logging, redaction, audit retention, dashboards, and production alerts

Chapter 9.2: Performance and Resilience Engineering

This chapter focuses on keeping AI applications responsive and resilient when external model providers or supporting services are slow or unavailable. You will learn asynchronous and parallel processing, streaming, timeouts, cancellation, retries with backoff and jitter, circuit breakers, bulkheads, provider failover, graceful degradation, deterministic fallbacks, concurrency control, backpressure, caching, idempotency, queues, and workload smoothing.

Highlighted Topics

- Async processing and parallel execution
- Streaming for perceived latency
- Timeouts and cancellation
- Retries with exponential backoff and jitter
- Circuit breakers and bulkheads
- Fallback models and provider failover
- Concurrency limits and backpressure
- Exact caching, semantic caching, distributed caching, and idempotency
- Queue-based workload smoothing
- Resilience strategies for external AI dependencies

- Graceful degradation and degraded-mode operation when AI dependencies are unavailable
- Deterministic or non-AI fallback paths for critical application workflows

Chapter 9.3: AI Gateway, Rate Limiting, and Cost Governance

This chapter teaches centralized control over AI consumption and cost. You will learn AI gateway concepts, centralized model access, user and tenant rate limits, token quotas, throttling, budget alerts, token and prompt optimization, embedding reuse, model routing, semantic caching, cost attribution, centralized authentication and authorization, and abuse prevention.

Highlighted Topics

- AI gateway and centralized policy concepts
- Centralized model access and governance
- Per-user and per-tenant request limits
- Token quotas and usage throttling
- Budget thresholds and alerts
- Token optimization, prompt compression, and response-length control
- Embedding reuse and batch processing
- Model routing based on cost and capability
- Semantic caching at the gateway
- Cost attribution, chargeback/showback, centralized authentication/authorization, and abuse prevention

Module 10: Deployment, Distributed AI, DevOps, and Production Operations

Bring the complete AI solution into production using modern .NET and Azure engineering practices. Participants will learn distributed application orchestration with .NET Aspire, containerization, Azure Container Registry, Azure deployment, Azure DevOps and Azure Pipelines, LLMOps, release management, and lifecycle practices for secure, maintainable, observable, and resilient enterprise AI systems.

Chapter 10.1: .NET Aspire and Distributed AI Applications

This chapter introduces .NET Aspire for composing and operating distributed AI applications. You will learn AppHost, Service Defaults, service discovery, configuration, secrets, worker and AI services, databases, vector stores, distributed caching, container resources, OpenTelemetry integration, local orchestration, and distributed RAG and background-processing architectures.

Highlighted Topics

- .NET Aspire architecture
- AppHost and Service Defaults
- Service discovery
- Configuration and secrets
- AI services and worker services
- Databases, vector stores, and distributed caching
- Container resources
- OpenTelemetry-based observability

- Local distributed-application orchestration
- Distributed RAG and background-processing architectures

Chapter 10.2: Docker and Containerization for AI Applications

This chapter teaches how to package AI-enabled .NET applications consistently using containers. You will learn Dockerfiles and multi-stage builds, Docker Compose, configuration and networking, secure secret handling, health/readiness/liveness checks, container security, local AI dependencies, and image optimization for production deployments.

Highlighted Topics

- Containerizing ASP.NET Core AI applications
- Dockerfile and multi-stage build concepts
- Docker Compose for dependent AI services
- Container configuration and networking
- Secure API keys and secrets in containers
- Health, readiness, and liveness checks
- Container security
- Local AI dependencies in containers
- Production image optimization

Chapter 10.3: Azure Deployment Architecture

This chapter maps the application architecture to Azure deployment choices. You will compare Azure App Service, Container Apps, and AKS; use Azure Container Registry for application images; integrate Key Vault, Managed Identity, Azure RBAC, Application Insights, Azure AI Search and vector stores; understand private networking and isolation; and plan scaling and environment-specific configuration from development through production.

Highlighted Topics

- Azure App Service
- Azure Container Apps
- AKS overview for AI workloads
- Azure Container Registry (ACR) for private container images
- Microsoft Entra ID and Azure RBAC for deployment-time and runtime access control
- Azure Key Vault and Managed Identity
- Application Insights
- Azure AI Search deployment considerations
- Vector-database hosting considerations
- Private endpoints and network-isolation overview
- Scaling and environment configuration
- Development, testing, staging, and production environments

Chapter 10.4: CI/CD for AI Applications Using Azure DevOps and Azure Pipelines

This chapter automates delivery of AI-enabled .NET applications using Azure DevOps and Azure Pipelines. You will learn Azure Repos and YAML pipelines, service

connections, build and test stages, AI evaluation and security gates, Docker image creation, Azure Container Registry, deployment to Azure App Service and Azure Container Apps, environment approvals, secure variables and secrets, smoke testing, rollback, and end-to-end release traceability across code, prompts, models, data, and container images.

Highlighted Topics

- Azure DevOps organization, project, and Azure Repos fundamentals
- Azure Pipelines and YAML pipeline structure
- Azure service connections and secure Azure authentication for pipelines
- Build, restore, test, and package stages for .NET AI applications
- AI evaluation and security checks as pipeline quality gates
- Docker image build, tagging, and versioning
- Azure Container Registry (ACR)
- Build and push Docker images to ACR from Azure Pipelines
- Deploy ACR images to Azure App Service
- Deploy ACR images to Azure Container Apps
- Development, testing, staging, and production pipeline environments
- Approvals, checks, variables, and secure secret handling
- Prompt and model-configuration versioning in the release process
- Smoke testing, rollback, deployment recovery, and release traceability

Chapter 10.5: LLMOps and AI Release Management

This chapter introduces LLMOps as the operational discipline for managing AI-specific assets and releases. You will learn versioning for prompts, models, evaluation datasets, knowledge bases, and indexes; release criteria and quality gates; staged rollout; continuous evaluation; regression monitoring for quality, cost, and performance; and rollback and incident response.

Highlighted Topics

- Prompt versioning
- Model configuration and snapshot versioning
- Evaluation dataset versioning
- Knowledge-base and index versioning
- Release approval criteria and quality gates
- Canary and staged rollout concepts
- Continuous evaluation
- AI quality regression detection
- Cost and performance regression monitoring
- Rollback and incident response

Chapter 10.6: Production AI Architecture and Lifecycle Management

This chapter ties the entire course together as a production architecture and lifecycle model. You will learn provider-independent, gateway/router, RAG, agent, background, event-driven, distributed, and multi-tenant architecture patterns; explicit service boundaries for prompts, retrieval, tools, evaluation, and auditing; security, observability, resilience, and cost by design; cloud/local/hybrid deployment; readiness

reviews; operational runbooks; lifecycle management; freshness; incident response; backup and continuous improvement.

Highlighted Topics

- Layered and provider-independent AI application architecture
- RAG service architecture
- Agent service architecture
- Background-processing and event-driven AI workflows
- Distributed AI architecture with .NET Aspire
- Multi-tenant AI architecture
- Security, observability, resilience, and cost governance by design
- Cloud, local, and hybrid deployment patterns
- Production readiness reviews and operational runbooks
- Model/provider lifecycle, prompt/retrieval configuration lifecycle, knowledge freshness, incident response, quality degradation monitoring, backup/recovery, and continuous improvement
- AI gateway, model-router, provider-adapter, and provider-fallback design patterns
- Prompt, retrieval, tool, evaluation, audit, and policy service boundaries
- Production design patterns for RAG, agents, background jobs, and event-driven AI workloads

Course Completion Outcome

By the end of the course, participants should be able to design, build, secure, evaluate, deploy, monitor, and operate professional AI-powered .NET applications while choosing appropriately between prompting, RAG, agents, fine-tuning, traditional Machine Learning, business rules, and local AI approaches.

Join Our Learning Community

🌐 **Website:** www.dotnettutorials.net
✉️ **Email:** info@dotnettutorials.net
📞 **WhatsApp/Call: +91 70218 01173**
💬 **WhatsApp Group:** Click here to join
📢 **Telegram Group:** @dotnettutorials
📺 **YouTube Channel:** Click here to join